Characterizing the Expressivity of Local Attention in Transformers

Transformer 中局部注意力表达能力的表征

Jiaoda Li Ryan Cotterell

李嘉达 Ryan Cotterell

{jiaoda.li, ryan.cotterell}@inf.ethz.ch

{jiaoda.li, ryan.cotterell}@inf.ethz.ch

ETH zürich

ETH Zurich

Abstract

摘要

The transformer is the most popular neural architecture for language modeling. The cornerstone of the transformer is its global attention mechanism, which lets the model aggregate information from all preceding tokens before generating the next token. One common variant of attention is called local attention, which restricts each token to aggregating information from a bounded window of predecessors, reducing the quadratic cost of global attention to linear. Although this restriction is usually motivated by efficiency, it has also been found to improve model quality, a phenomenon that has so far lacked a satisfactory explanation. We provide a formal account of this phenomenon in terms of recognizer expressivity. It has been shown that fixed-precision transformers with global attention correspond to a fragment of linear temporal logic containing a single past operator. We additionally prove that adding local attention introduces a second temporal operator, strictly enlarging the class of recognizable regular languages. Moreover, global and local attention are expressively complementary: neither subsumes the other, and combining them yields the richest fragment. Experiments on formal language recognition and natural language modeling corroborate the theory, showing that hybrid global–local transformers outperform their global-only counterparts.

变压器是语言建模中最受欢迎的神经架构。变压器的核心在于其全局注意力机制，该机制允许模型在生成下一个标记之前，从所有先前的标记中聚合信息。一种常见的注意力变体称为局部注意力，它限制每个标记只能从有限的前缀窗口中聚合信息，从而将全局注意力的二次成本降低到线性。尽管这种限制通常出于效率的考虑，但研究发现它也有助于提高模型的质量，这种现象至今尚缺乏令人满意的解释。我们从识别器表达力的角度提供了这一现象的正式解释。研究表明，具有全局注意力的固定精度变压器对应于包含一个过去操作符的线性时态逻辑的一个片段。此外，我们还证明，添加局部注意力会引入第二个时态操作符，从而严格扩大了可识别的正则语言的类别。此外，全局注意力和局部注意力在表达力上是互补的：它们彼此并不包含，结合它们可以得到最丰富的片段。在形式语言识别和自然语言建模上的实验验证了这一理论，表明混合全局-局部变压器优于仅使用全局注意力的模型。

1 Introduction

1 引言

The transformer (Vaswani et al., 2017) is the dominant neural architecture in modern language modeling (Radford et al., 2018, 2019; Brown et al., 2020; OpenAI, 2023). At its core lies the attention mechanism (Schmidhuber, 1992; Bahdanau et al., 2015), which allows the model to aggregate

information from all previously generated tokens before generating the next one. Local attention (Luong et al., 2015; Child et al., 2019; Beltagy et al., 2020; Zaheer et al., 2020) is a popular attention variant that restricts the model to aggregating over a fixed-size window of preceding tokens.

变压器（Vaswani 等人，2017）是现代语言建模中占主导地位的神经架构（Radford 等人，2018, 2019; Brown 等人，2020; OpenAI, 2023）。其核心是注意力机制（Schmidhuber, 1992; Bahdanau 等人，2015），该机制允许模型在生成下一个 token 之前，从所有先前生成的 token 中聚合信息。局部注意力（Luong 等人，2015; Child 等人，2019; Beltagy 等人，2020; Zaheer 等人，2020）是一种流行的注意力变体，它限制模型仅在固定大小的先前 token 窗口中进行信息聚合。

Local attention is typically motivated by efficiency. Standard global attention aggregates over all previous positions, incurring quadratic cost in sequence length; local attention reduces this to linear. At first glance, restricting the range of tokens that a transformer can attend to appear to be a strict weakening—a trade-off of expressivity for speed. Surprisingly, however, local attention has been repeatedly observed to yield empirical improvements, both in machine translation (Luong et al., 2015) and in language modeling (Child et al., 2019; Beltagy et al., 2020; Zaheer et al., 2020)—even when computational budgets are held constant.

局部注意力通常由效率驱动。标准的全局注意力会聚合所有先前的位置，导致序列长度的二次成本；局部注意力将这一成本降低到线性。乍一看，限制变压器可以关注的标记范围似乎是一种严格的削弱——以速度换取表达能力的权衡。然而，令人惊讶的是，局部注意力在机器翻译（Luong 等人，2015）和语言建模（Child 等人，2019；Beltagy 等人，2020；Zaheer 等人，2020）中反复被观察到能带来实证改进——即使在计算预算保持不变的情况下也是如此。

We resolve this puzzle by analyzing which formal languages different attention patterns can recognize, a standard lens in the formal study of neural models (Delétang et al., 2023; Butoi et al., 2025). This lens also has practical relevance: modern language models (OpenAI, 2023; DeepSeek-AI, 2025; Team et al., 2025) are routinely used to perform computation via prompting (Wei et al., 2022; Li et al., 2024; Merrill and Sabharwal, 2024). In this vein, Li and Cotterell (2025) show that fixed-precision transformer language models with global attention correspond exactly to the fragment of linear temporal logic with a single past operator. Expanding their work, we prove that adding local attention introduces a new temporal operator, enabling the recognition of a strictly larger class of regular languages. These include the locally testable languages, a family closely connected to n-gram models and extensively studied in formal language theory (McNaughton and Papert, 1971, Ch. 2). Crucially, local attention is complementary rather than a drop-in replacement for global attention: with purely local attention, the computation depends only on a fixed-length suffix of the input, collapsing expressivity to the smaller class of definite languages (Kleene, 1956, §5).

我们通过分析不同的注意力模式能够识别哪些形式语言来解决这个谜题，这是神经模型形式研究中的一个标准视角（Delétang 等人，2023；Butoi 等人，2025）。这种视角也具有实际意义：现代语言模型（OpenAI，2023；DeepSeek-AI，2025；Team 等人，2025）通常通过提示（prompting）来进行计算（Wei 等人，2022；Li 等人，2024；Merrill 和 Sabharwal，2024）。沿着这个思路，Li 和 Cotterell（2025）表明，具有全局注意力的固定精度变换器语言模型正好对应于线性时态逻辑中带有单个过去运算符的片段。扩展他们的工作，我们证明添加局部注意力引入了一个新的时间运算符，使得能够识别更大类的正则语言。这些语言包括局部可测试语言，这是一种与 n-gram 模型密切相关且在形式语言理论中被广泛研究的语言家族（McNaughton 和 Papert，1971，第 2 章）。关键的是，局部注意力是与全局注意力互补的，而不是简单的替代：仅使用纯局部注意力时，计算仅依赖于输入的固定长度后缀，这将表达能力压缩到更小的确定语言类（Kleene，1956，第 5 节）。

Empirically, we test these predictions on formal language recognition with length generalization. We compare transformers with (i) global attention, (ii) local attention, and (iii) a hybrid that mixes global and local attention. The results align with the theory: hybrid models recognize strictly more languages than either extreme and generalize more reliably to longer sequences. Varying local attention's window size further shows that a window of size one is consistently the strongest choice in the local-attention family, both for purely local models and for hybrid models. Finally, we evaluate models equipped with two popular positional encodings—sinusoidal (SiPE) and rotary (RoPE)—and find that neither compensates for the absence of local attention. We fur-

ther show that these findings extend to natural language: on WikiText-2, hybrid attention with a window of size one consistently achieves the lowest perplexity across positional encodings.

从实证角度来看，我们在形式语言识别任务中测试了这些预测，该任务涉及长度泛化。我们比较了具有以下三种注意力机制的变压器模型：(i) 全局注意力，(ii) 局部注意力，以及 (iii) 混合注意力（结合全局和局部注意力）。实验结果与理论一致：混合模型能够识别比极端情况更多的语言，并且在更长序列上具有更可靠的泛化能力。进一步改变局部注意力的窗口大小表明，在局部注意力家族中，窗口大小为 1 的选择在纯局部模型和混合模型中都是一致的最佳选择。最后，我们评估了配备两种流行位置编码的模型——正弦（SiPE）和旋转（RoPE）编码，并发现这两种编码都无法弥补局部注意力的缺失。我们进一步表明，这些发现也适用于自然语言：在 WikiText-2 数据集上，窗口大小为 1 的混合注意力在所有位置编码中始终实现最低的困惑度。

2 Linear Temporal Logic

2 线性时态逻辑

In this section, we introduce linear temporal logic, the main tool we use to characterize the expressive power of transformers under different types of attention. We examine the fragments relevant to our analysis, prove the separations between them, and connect them to classical characterizations from algebraic formal language theory.

在本节中，我们介绍线性时态逻辑，这是我们用来刻画在不同类型的注意力下变换器的表达能力的主要工具。我们考察与我们的分析相关的片段，证明它们之间的分离，并将它们与代数形式语言理论中的经典刻画相连接。

2.1 Preliminaries

2.1 基础知识

We now introduce linear temporal logic (LTL; Kamp, 1968; Pnueli, 1977), which has become a popular tool in the analysis of transformers (Yang et al., 2024, 2026; Li and Cotterell, 2025).

我们现在介绍线性时态逻辑（LTL；Kamp, 1968; Pnueli, 1977），它已成为分析转换器的流行工具（Yang 等人，2024, 2026; Li 和 Cotterell, 2025）。

Tokens, Strings and Languages. An alphabet is a finite, non-empty set of tokens. Fix an alphabet Σ. A string over Σ is a finite sequence of tokens drawn from Σ. The length of a string $w = w_1 \cdots w_N$, denoted $|w| = N$, is the number of tokens it contains. We write Σ^* for the set of all strings over Σ, and call any subset $L \subseteq \Sigma^*$ a language.

符号、字符串和语言。一个字母表是一个有限的、非空的符号集合。固定一个字母表 Σ。一个在 Σ 上的字符串是一个从 Σ 中选取的有限符号序列。一个字符串 $w = w_1 \cdots w_N$ 的长度，记作 $|w| = N$，是它所包含的符号数目。我们用 Σ^* 表示所有在 Σ 上的字符串集合，并将任何子集 $L \subseteq \Sigma^*$ 称为一种语言。

Linear Temporal Logic. The syntax and semantics of linear temporal logic (LTL[P, Y, S, U]) as well as the definition of operator depth are recalled in §A. We also give a definition of what it means for a formula in LTL to define a language; see §A.3. Given a formula ψ, we write $\mathbb{L}(\psi)$ for the set of all strings satisfying ψ.

线性时态逻辑。线性时态逻辑（LTL[P, Y, S, U]）的语法和语义，以及运算符深度的定义在 §A 中被回顾。我们还给出了一个关于 LTL 公式定义语言的定义；参见 §A.3。给定一个公式 ψ，我们用 $\mathbb{L}(\psi)$ 表示满足 ψ 的所有字符串的集合。

Fragments of LTL[P, Y, S, U]. For a set of temporal operators $\mathcal{O} \subseteq \{\mathbf{P}, \mathbf{Y}, \mathbf{S}, \mathbf{U}\}$, we write

LTL[P, Y, S, U] 的片段。对于一组时态运算符 $\mathcal{O} \subseteq \{\mathbf{P}, \mathbf{Y}, \mathbf{S}, \mathbf{U}\}$ ，我们写成

LTL[O] for the corresponding fragment in which only operators from O are allowed.

LTL[O] 对应的片段，其中只允许使用 O 中的运算符。

Definition 2.1. Let $\mathcal{O} \subseteq \{\mathbf{P}, \mathbf{Y}, \mathbf{S}, \mathbf{U}\}$ be a set of temporal operators. A language L is definable by LTL[$\mathcal{O}$] if there exists a formula ψ in LTL[$\mathcal{O}$] such that $\mathbb{L}(\psi) = L$.

定义 2.1. 设 $\mathcal{O} \subseteq \{\mathbf{P}, \mathbf{Y}, \mathbf{S}, \mathbf{U}\}$ 是一个时态运算符的集合。一个语言 L 可以由 LTL[$\mathcal{O}$] 定义，如果存在一个公式 ψ 属于 LTL[$\mathcal{O}$]，使得 $\mathbb{L}(\psi) = L$ 。

Gabbay et al. (1980) show that every language definable by LTL[P, Y, S, U] is also definable by both LTL[S] and LTL[U]; in the paper, we write LTL[S] for full linear temporal logic. The fragments LTL [P] and LTL [Y] are strictly less expressive, i.e., there exist languages definable by LTL[S] that are not definable by either LTL [P] or LTL [Y].

Gabbay 等人（1980）表明，每个可由 LTL[P, Y, S, U] 定义的语言也可以由 LTL[S] 和 LTL[U] 同时定义；在本文中，我们将 LTL[S] 作为完整的线性时态逻辑。片段 LTL[P] 和 LTL[Y] 的表达能力严格较低，即存在一些可由 LTL[S] 定义的语言，却无法由 LTL[P] 或 LTL[Y] 定义。

2.2 Characterizing LTL [Y]

2.2 描述 LTL [Y]

We first characterize the expressive power of LTL [Y] by relating it to a classical class of regular languages. Because formulas in LTL [Y] can only look back a bounded number of steps, we show they naturally correspond to languages determined by a bounded suffix—the definite languages.

我们首先通过将其与一个经典的正则语言类进行比较，来刻画 LTL [Y] 的表达能力。由于 LTL [Y] 中的公式只能回溯有限数量的步骤，我们证明它们自然地对应于由有限后缀决定的语言——即确定性语言。

For a string w of length N and an integer $m \leq N$, an m -factor of w is any contiguous substring of w of length m . The m -prefix of w is its initial substring of length m , and the m -suffix of w is its final substring of length m . We write $\mathbb{I}_m(\mathsf{w})$ for the set of all m -factors of w , and $P_m(\mathsf{w})$ and $S_m(\mathsf{w})$ for the m -prefix and m -suffix of w , respectively. If $m > |\mathsf{w}|$, we set $P_m(\mathsf{w}) = S_m(\mathsf{w}) = \mathsf{w}$. Formally, a language $L \subseteq \Sigma^*$ is m -definite, for some $m \geq 0$, if L is a Boolean combination of languages of the form $\{\mathsf{w} \in \Sigma^* \mid S_m(\mathsf{w}) = \mathsf{u}\}$. A language is definite if it is m -definite for some $m \geq 0$.

对于一个长度为 N 的字符串 w 和一个整数 $m \leq N$ ，字符串 w 的一个 m -因子是指其长度为 m 的任意连续子串。字符串 w 的 m -前缀是指其长度为 m 的初始子串，而字符串 w 的 m -后缀是指其长度为 m 的末尾子串。我们用 $\mathbb{I}_m(\mathsf{w})$ 表示字符串 w 的所有 m -因子的集合，用 $P_m(\mathsf{w})$ 和 $S_m(\mathsf{w})$ 分别表示字符串 w 的 m -前缀和 m -后缀。如果 $m > |\mathsf{w}|$ ，我们设置 $P_m(\mathsf{w}) = S_m(\mathsf{w}) = \mathsf{w}$ 。形式上，一个语言 $L \subseteq \Sigma^*$ 是 m -确定的，当且仅当对于某个 $m \geq 0$ ，L 是形如 $\{\mathsf{w} \in \Sigma^* \mid S_m(\mathsf{w}) = \mathsf{u}\}$ 的语言的布尔组合。一个语言是确定的，当且仅当它对于某个 $m \geq 0$ 是 m -确定的。

We next relate LTL [Y] to definite languages, for which we also provide algebraic and automata-theoretic characterizations. The relevant background, including definitions of the syntactic semigroups and deterministic finite automata (DFAs), are recalled in §B.

我们接下来将 LTL [Y] 与确定性语言相关联，对于这些语言，我们也提供了代数和自动机理论的特征描述。相关背景，包括语法半群和确定性有限自动机（DFAs）的定义，参见 §B。

Theorem 2.2. Let $L \subseteq \Sigma^*$ be a regular language, let $\mathbb{S}$ be its syntactic semigroup, and let $\mathcal{A}$ be the minimal DFA recognizing L . The following are equivalent:

定理 2.2. 设 $L \subseteq \Sigma^*$ 为一个正则语言，设 $\mathbb{S}$ 为其语法半群，设 $\mathcal{A}$ 为识别 L 的最小 DFA。以下各项等价：

1. L is definable in LTL [Y];
2. L is definite;
3. every idempotent $e \in \mathbb{S}$ is a right zero, i.e., for all $s \in \mathbb{S}$, $se = e$;
4. every transformation induced by a non-empty string is non-permutational.
5. L 可以在 LTL [Y] 中定义；
6. L 是确定的；
7. 每个幂等的 $e \in \mathbb{S}$ 是一个右零元，即对于所有 $s \in \mathbb{S}$，都有 $se = e$；
8. 每个由非空字符串诱导的变换都是非置换的。

Proof. See §C.1.

证明。参见 §C.1。

2.3 LTL [P] and LTL [Y] are Incomparable

2.3 LTL [P] 和 LTL [Y] 是不可比较的

Li and Cotterell (2025) give a rich characterization of LTL [P]. Building on this and §2.2, we now relate LTL [P] and LTL [Y]. However, we first discuss what it means to compare two sets of operators for linear temporal logic. Fix $O_1, O_2 \subseteq \{P, Y, S, U\}$. We say $LTL[O_1]$ is more expressive than $LTL[O_2]$, written in symbols $LTL[O_2] \subseteq LTL[O_1]$, if every language definable in $LTL[O_2]$ is also definable in $LTL[O_1]$. We say $LTL[O_1]$ is strictly more expressive than $LTL[O_2]$, written in symbols as $LTL[O_2] \subset LTL[O_1]$, if $LTL[O_2] \subseteq LTL[O_1]$ and there exists a language definable in $LTL[O_1]$ but not in $LTL[O_2]$. We call $LTL[O_1]$ and $LTL[O_2]$ incomparable if neither is more expressive than the other.

李和科特雷尔（2025）对 LTL [P] 进行了丰富的刻画。在借鉴这一成果以及第 2.2 节的基础上，我们现在将 LTL [P] 与 LTL [Y] 进行关联。然而，我们首先讨论如何比较线性时态逻辑中的两个运算符集合。固定 $O_1, O_2 \subseteq \{P, Y, S, U\}$。我们说 $LTL[O_1]$ 比 $LTL[O_2]$ 更具表达力，记作 $LTL[O_2] \subseteq LTL[O_1]$，如果在 $LTL[O_2]$ 中可定义的每一语言也都可以在 $LTL[O_1]$ 中定义。我们说 $LTL[O_1]$ 严格地比 $LTL[O_2]$ 更具表达力，记作 $LTL[O_2] \subset LTL[O_1]$，如果 $LTL[O_2] \subseteq LTL[O_1]$ 且存在一个可在 $LTL[O_1]$ 中定义但不能在 $LTL[O_2]$ 中定义的语言。我们称 $LTL[O_1]$ 和 $LTL[O_2]$ 是不可比较的，如果其中任何一个都不比另一个更具表达力。

Proposition 2.3. There exists a language definable in LTL [P] that is not definable in LTL [Y].

命题 2.3. 存在一个语言可以定义在 LTL [P] 中，但不能定义在 LTL [Y] 中。

Proof. Consider the language $a\Sigma^*$, i.e., the set of strings whose first token is a. It is definable in LTL [P] by $\mathbf{P}(\pi_a \wedge \neg\mathbf{P}\top)$, i.e., $\mathbb{L}(\mathbf{P}(\pi_a \wedge \neg\mathbf{P}\top)) = a\Sigma^*$. However, it is not definite, and therefore not definable in LTL [Y] by Theorem 2.2.

证明。考虑语言 $a\Sigma^*$，即所有以 a 作为第一个词的字符串的集合。它可以在 LTL [P] 中通过 $\mathbf{P}(\pi_a \wedge \neg\mathbf{P}\top)$ 定义，即 $\mathbb{L}(\mathbf{P}(\pi_a \wedge \neg\mathbf{P}\top)) = a\Sigma^*$。然而，它不是确定性的，因此根据定理 2.2，它不能在 LTL [Y] 中定义。

Proposition 2.4. There exists a language definable in LTL [Y] that is not definable in LTL [P].

命题 2.4. 存在一个可以在 LTL [Y] 中定义的语言，而无法在 LTL [P] 中定义。

Proof. Consider the language $\Sigma^*\mathsf{a}$, i.e., the set of strings whose last token is a . It is definable in LTL [Y] by $\mathbf{Y}\pi_a$, i.e., $\mathbb{L}(\mathbf{Y}\pi_a) = \Sigma^*\mathsf{a}$. However, it is not definable in LTL [P]: Li and Cotterell (2025) show that LTL [P] defines exactly the class of left-deterministic polynomials, and $\Sigma^*\mathsf{a}$ is not a left-deterministic polynomial. See §B for an exposition of the relevant technical definitions.

证明。考虑语言 $\Sigma^*\mathsf{a}$，即所有最后一个词是 a 的字符串集合。该语言可以用 LTL [Y] 定义，例如通过 $\mathbf{Y}\pi_a$，即 $\mathbb{L}(\mathbf{Y}\pi_a) = \Sigma^*\mathsf{a}$。然而，该语言不能用 LTL [P] 定义：Li 和 Cotterell（2025）表明，LTL [P] 定义的恰好是左确定多项式类，而 $\Sigma^*\mathsf{a}$ 不是一个左确定多项式。有关相关技术定义的阐述，请参见 §B。

Corollary 2.5. The fragments LTL [P] and LTL [Y] are incomparable.

推论 2.5. 片段 LTL [P] 和 LTL [Y] 是不可比较的。

Proof. This is a direct implication of both Propositions 2.3 and 2.4.

证明。这是命题 2.3 和 2.4 的直接推论。

2.4 Characterizing LTL [P, Y]

2.4 描述 LTL [P, Y]

We give another implication of Corollary 2.5 below.

我们给出推论 2.5 的另一个推论。

Corollary 2.6. LTL [Y], LTL [P] ⊂ LTL [P, Y].

推论 2.6. LTL [Y], LTL [P] ⊂ LTL [P, Y].

Proof. Note that LTL [P, Y] trivially contains both P and Y; it subsumes both fragments. However, because LTL [P] and LTL [Y] are incomparable by Corollary 2.5, there exists a language definable in LTL [P, Y] that is not definable in LTL [P] as well as a language definable in LTL [P, Y] that is not

证明。注意 LTL [P, Y] 显然包含 P 和 Y，它涵盖了这两个片段。然而，由于根据推论 2.5，LTL [P] 和 LTL [Y] 是不可比较的，因此存在一些语言可以被 LTL [P, Y] 定义，但不能被 LTL [P] 定义，同时还有其他语言可以被 LTL [P, Y] 定义，但不能被 LTL [Y] 定义。

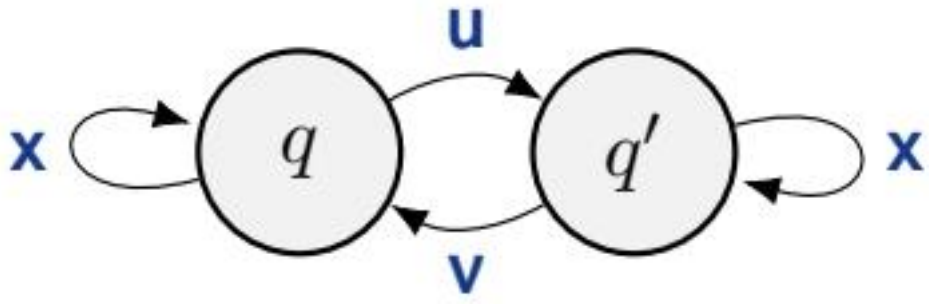

Figure 1: images/1fba54df6916c86961d934869ef060cf1e36be9c151fd4ec3ba07b0ddfd6812f.jpg

Figure 1: Forbidden configuration in the minimal DFAs of LTL [P, Y]-definable languages.

图 1：LTL [P, Y]-可定义语言的最小 DFA 中的禁止配置。

definable in LTL [Y]. This renders both subsumptions strict, as we sought to show.

可在 LTL [Y] 中定义。这使得两种子集关系都是严格的，正如我们所希望展示的那样。

To further characterize the expressive power of LTL [P, Y], we relate it to standard algebraic and automata-theoretic notions. The relevant definitions, including locally R-trivial semigroups and configurations of DFAs, are recalled in §B.

为进一步刻画 LTL [P, Y] 的表达能力，我们将它与标准的代数和自动机理论概念联系起来。相关的定义，包括局部 R-平凡半群和确定性有限自动机的配置，参见 §B。

Theorem 2.7. Let $L \subseteq \Sigma^*$ be a regular language, $\mathbb{S}$ its syntactic semigroup, and $\mathcal{A}$ the minimal DFA recognizing L . The following are equivalent:

定理 2.7. 设 $L \subseteq \Sigma^*$ 是一个正则语言，$\mathbb{S}$ 是它的语法半群，$\mathcal{A}$ 是识别 L 的最小 DFA。以下条件等价：

1. L is definable in LTL [P, Y];
2. $\mathbb{S}$ is locally $\mathcal{R}$ -trivial;
3. $\mathcal{A}$ contains no configuration of the form shown in Fig. 1.
4. L 在 LTL [P, Y] 中是可定义的；
5. $\mathbb{S}$ 是局部 $\mathcal{R}$ -平凡的；
6. $\mathcal{A}$ 不包含图 1 中所示形式的配置。

Proof. See §C.2.

证明。参见 §C.2。

Next, we give two examples that respectively illustrate a language definable in LTL [P, Y] and a language beyond its expressive power.

接下来，我们给出两个例子，分别说明一个可在 LTL [P, Y] 中定义的语言和一个超出其表达能力的语言。

2.4.1 The Locally Testable Languages

2.4.1 可局部检验的语言

The addition of Y enables the definition of classes of languages that are linguistically relevant yet lie beyond the expressive power of LTL [P] alone. As a representative example, we consider the locally testable languages.

加入 Y 后，可以定义出一些在语言学上具有重要意义但超越了 LTL [P] 表达能力的语言类。作为一个代表性的例子，我们考虑局部可测试语言。

A language $L \subseteq \Sigma^*$ is locally m-testable, for some m > 0, if it is a Boolean combination of lan-

guages of the following three forms:

一种语言 $L \subseteq \Sigma^*$ 在某个 m > 0 的情况下是局部 m-可测试的，如果它是以下三种形式语言的布尔组合：

$$\{\mathbf{w} \in \Sigma^* \mid \mathbf{u} \in \mathbb{I}_m(\mathbf{w})\}, \qquad \mathbf{u} \in \Sigma^m, \tag{1a}$$

$$\{\mathbf{w} \in \Sigma^* \mid P_{m-1}(\mathbf{w}) = \mathbf{u}\}, \quad \mathbf{u} \in \Sigma^{m-1}, \quad (1\mathrm{b})$$

$$\{\mathbf{w} \in \Sigma^* \mid S_{m-1}(\mathbf{w}) = \mathbf{u}\}, \quad \mathbf{u} \in \Sigma^{m-1}. \quad (1c)$$

A language is locally testable if it is locally m-testable for some m > 0. Li and Cotterell (2025) show that LTL [P] cannot define many simple locally testable languages, e.g., $\Sigma^* a$ and $\Sigma^* ab\Sigma^*$, because they are not right-deterministic. In contrast, as we show in the subsequent theorem, all locally testable languages are definable in LTL [P, Y], yielding a witness to the separation in expressive power between LTL [P] and LTL [P, Y].

一种语言是局部可测试的（locally testable），如果它对于某个 m > 0 是局部 m-可测试的。Li 和 Cotterell（2025）表明，LTL [P] 无法定义许多简单的局部可测试语言，例如 $\Sigma^* a$ 和 $\Sigma^* ab\Sigma^*$，因为它们不是右确定性的。相反，如我们在后续定理中所展示的，所有局部可测试语言都可以在 LTL [P, Y] 中定义，这提供了一个证据，说明 LTL [P] 和 LTL [P, Y] 在表达能力上的分离。

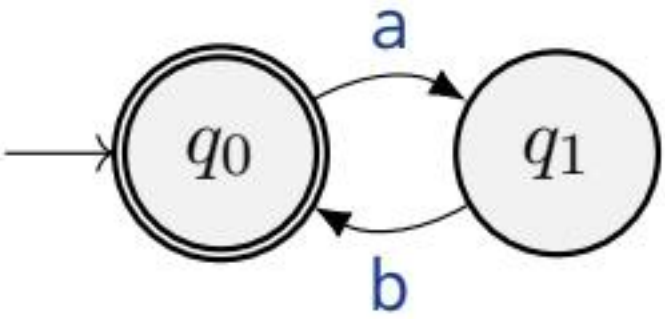

Figure 2: images/9b2518a958412bdcbafeeaa2cfddba70346e0da61d78e2f30d3d8ce0f6f0f3df.jpg

(a) (ab)*

(a) (ab)*

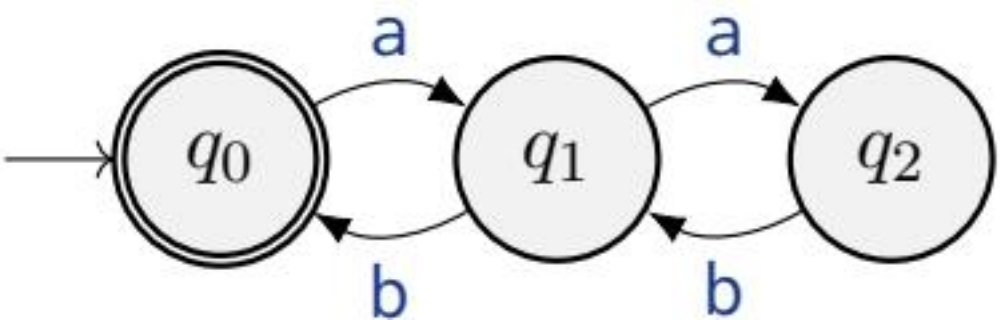

Figure 3: images/c42e9ffcf14d11cc72437e35f3e5f906bf8fd5d7c76e6e3270280c25fff88aae.jpg

(b) $(a(ab)^*b)^*$

Figure 2: Minimal DFAs. Nodes represent states and arrows represent transitions. The initial

state is indicated by an incoming arrow with no source node, and accepting states are shown with double circles.

(b) $(a(ab)^*b)^*$
图 2：最小化 DFA。节点表示状态，箭头表示转移。初始状态通过没有源节点的入射箭头表示，接受状态则用双圆圈表示。

Theorem 2.8. Any locally testable language is definable in LTL [P, Y].

定理 2.8. 任何局部可测试语言都可以在 LTL [P, Y] 中定义。

Proof. See §C.3.

证明。参见 §C.3。

2.4.2 A Bounded Dyck Language

2.4.2 有界 Dyck 语言

Despite this gain in expressivity, LTL [P, Y] still defines a limited class of languages. For instance, it does not define all star-free languages, which are exactly the languages definable in LTL[S]. One illustrative family is the bounded Dyck languages, consisting of well-balanced parentheses with a fixed upper bound on nesting depth. [1] The bounded Dyck language of depth k consists of all well-balanced strings whose maximum nesting depth is at most k . [2] For example, over $\{a, b\}$, where a is an opening parenthesis and b is a closing parenthesis, the languages

尽管在表达能力方面有所提升，LTL [P, Y] 仍然只能定义有限类的语言。例如，它并不能定义所有的星自由语言（star-free languages），而这些语言正是由 LTL[S] 所定义的。一个具有代表性的语言家族是受限的 Dyck 语言（bounded Dyck languages），它由具有固定嵌套深度上限的平衡括号序列组成。[1] 深度为 k 的受限 Dyck 语言包含所有最大嵌套深度不超过 k 的平衡字符串。[2] 例如，在 $\{a, b\}$ 上，其中 a 是一个左括号，b 是一个右括号，这些语言

$$(ab)^* \quad \text{and} \quad (a(ab)^*b)^* \tag{2}$$

correspond to maximum nesting depth 1 and 2.

对应最大嵌套深度 1 和 2。

The minimal DFAs for these languages are shown in Fig. 2. The depth-1 language (ab)* is recognizable by LTL [P, Y], because its minimal DFA avoids the forbidden configuration in Fig. 1. In contrast, the depth-2 language (a(ab)*b)* is not LTL [P, Y]-definable: its minimal DFA contains the configuration from Fig. 1, witnessed by $q = q_0$, $q' = q_1$, u = a, v = b, and x = ab.

这些语言的最小 DFA 如图 2 所示。深度为 1 的语言 (ab)* 可以由 LTL [P, Y] 识别，因为其最小 DFA 避免了图 1 中的禁止配置。相比之下，深度为 2 的语言 (a(ab)*b)* 不是 LTL [P, Y] 可定义的：其最小 DFA 包含了图 1 中的配置，由 $q = q_0$ ，$q' = q_1$ ，u = a，v = b，和 x = ab 所见证。

2.5 Characterizing LTL $[P, Y^{\leq k}]$

2.5 描述 LTL $[P, Y^{\leq k}]$

We now introduce two additional temporal operators derived from Y. The bounded operator $Y^{\leq k}$

我们现在引入两个额外的时态运算符，它们源自 Y。有界运算符 $Y^{\leq k}$

asks whether ψ holds at some position among the previous k steps, while the unbounded operator Y^* asks whether ψ holds at any earlier position. Formally, these two operators are defined as follows

询问 ψ 是否在前 k 步中的某个位置成立，而无界算子 Y^* 则询问 ψ 是否在任何更早的位置成立。形式上，这两个算子定义如下：

$$\mathbf{Y}^{\leq k}\psi \stackrel{\text{def}}{=} \bigvee_{i=1}^{k} \mathbf{Y}^i\psi = \bigvee_{i=1}^{k} \underbrace{\mathbf{Y}\mathbf{Y}\cdots\mathbf{Y}}_{i\text{times}}\psi \tag{3a}$$

$$\mathbf{Y}^{*}\psi \stackrel{\text{def}}{=} \bigvee_{i=1}^{\infty} \mathbf{Y}^i\psi = \bigvee_{i=1}^{\infty} \underbrace{\mathbf{Y}\mathbf{Y}\cdots\mathbf{Y}}_{i\text{times}}\psi \tag{3b}$$

Note that Y is the special case $Y^{\leq 1}$, and Y^* coincides with P. This last fact implies that $LTL[P, Y^*] = LTL[P]$, i.e., the fragments are not only comparable, they are equivalent. However, for any $k \geq 1$, the fragment $LTL[Y^{\leq k}]$, as is the case with LTL[Y], is incomparable with LTL[P]. We make this claim precise in the following theorem.

请注意，Y 是特殊情况 $Y^{\leq 1}$，并且 Y^* 与 P 相同。这一事实意味着 $LTL[P, Y^*] = LTL[P]$，即这些片段不仅可比，而且是等价的。然而，对于任何 $k \geq 1$，片段 $LTL[Y^{\leq k}]$，正如 LTL[Y] 的情况一样，与 LTL[P] 是不可比较的。我们在下面的定理中对这一说法进行了精确表述。

Theorem 2.9. For any $k \geq 1$, the fragments LTL [P] and LTL $\left[\mathbf{Y}^{\leq k}\right]$ are incomparable.

定理 2.9。对于任何 $k \geq 1$ ，片段 LTL [P] 和 LTL $\left[\mathbf{Y}^{\leq k}\right]$ 是不可比较的。

Proof. See §C.4.

证明。参见 §C.4。

Figure 4: images/5330aa9d6ce8c468d4dfcc53fee79f1f6f6a2a97835d27053c30014a704768b6.jpg

Thus, we arrive at the following corollary.

因此，我们得出以下推论。

Corollary 2.10. The fragment LTL $\left[\mathbf{P}, \mathbf{Y}^{\leq k}\right]$ is strictly more expressive than each of the fragments LTL [P] and LTL $\left[\mathbf{Y}^{\leq k}\right]$.

推论 2.10. 片段 LTL $\left[\mathbf{P}, \mathbf{Y}^{\leq k}\right]$ 的表达能力严格强于每个片段 LTL [P] 和 LTL $\left[\mathbf{Y}^{\leq k}\right]$ 。

Proof. See §C.5.

证明。参见 §C.5。

Figure 5: images/cd8be14987c58586bb8c9c6f43a8520a6c79ac708df38db8ce9bbda22f912656.jpg

We next analyze expressivity for various $k \geq 1$; we find $Y^{\leq k}$ is maximally expressive at k = 1.

我们接下来分析不同 $k \geq 1$ 的表达能力；我们发现 $Y^{\leq k}$ 在 k = 1 时具有最大的表达能力。

Proposition 2.11. The fragment LTL $[Y^{\leq k}]$ is strictly less expressive than LTL [Y] for k > 1.

命题 2.11. 片段 LTL $[Y^{\leq k}]$ 的表达能力严格弱于 LTL [Y]，当 k > 1 时。

Proof. See §C.6.

证明。参见 §C.6。

Figure 6: images/13ed97c59edfe4bfed566013e77067de89b3e00b7ac5366d8e15e1aae19259aa.jpg

We now show adding P preserves this strictness.

我们现在展示添加 P 保持了这种严格性。

Proposition 2.12. The fragment LTL $[P, Y^{\leq k}]$ is strictly less expressive than LTL $[P, Y]$ for k > 1.

命题 2.12. 片段 LTL $[P, Y^{\leq k}]$ 在 k > 1 时，严格弱于 LTL $[P, Y]$。

Proof. See §C.7.

证明。见 §C.7。

Figure 7: images/ba6c9e3e83143098a3c2e14c425c55b485f262bbff7903aaec8aa63badac27e5.jpg

The gap in expressivity between LTL $[P, Y \leq k]$ and LTL $[P, Y]$ can, however, be closed in the presence of suitable numerical predicates. Numerical predicates depend only on positions, and not on the tokens occurring at those positions (Straubing, 1994, II.2). A particularly important family is modular predicates MOD_m^r , with m > 0 and $r \geq 0$:

然而，在存在适当的数值谓词的情况下，LTL $[P, Y \leq k]$ 和 LTL $[P, Y]$ 之间的表达能力差距可以弥补。数值谓词仅依赖于位置，而不是这些位置上出现的符号（Straubing, 1994, II.2）。一个特别重要的谓词族是模数谓词 MOD_m^r ，其中 m > 0 和 $r \geq 0$：

$$\mathrm{w}, n \vDash \mathrm{MOD}_m^r \quad \text{iff} \quad n \equiv r \pmod{m}. \tag{4}$$

Proposition 2.13. For every k > 1, if modular predicates MOD_m^r are available for some $m \geq k$

, then LTL $[Y^{\leq k}]$ has the same expressive power as LTL [Y], and LTL $[P, Y^{\leq k}]$ has the same expressive power as LTL [P, Y].

命题 2.13。对于每一个 k > 1，如果对于某些 $m \geq k$ 可以获得模态谓词 MOD_m^r，那么 LTL $[Y^{\leq k}]$ 的表达能力与 LTL [Y] 相同，LTL $[P, Y^{\leq k}]$ 的表达能力与 LTL [P, Y] 相同。

Proof. See §C.8.

证明。参见 §C.8。

3 The Transformer

3 注意力机制

In this section, we formally define the transformer architecture we consider. Note that we analyze the transformer as a language recognizer.

在本节中，我们正式定义我们考虑的 transformer 架构。请注意，我们将 transformer 分析为一个语言识别器。

3.1 Preliminaries

3.1 基础知识

All arithmetic in this section is carried out over a set of representable numbers $\mathbb{F}$; we defer the precise formalization to §3.4. For now, the reader may assume that all relevant operations (addition, multiplication, exp, etc.) are well-defined over $\mathbb{F}$.

本节中的所有算术运算都是在一组可表示的数 $\mathbb{F}$ 上进行的；我们将在第 3.4 节中给出精确的形式化定义。目前，读者可以假设所有相关的运算（加法、乘法、指数运算等）在 $\mathbb{F}$ 上都有定义。

Definition 3.1 (Recognition). Fix an alphabet Σ . A language recognizer is a function $o : \Sigma^* \to \{0, 1\}$. A language $L \subseteq \Sigma^*$ is recognized by o if $o(\mathsf{w}) = 1$ for all $\mathsf{w} \in L$ and $o(\mathsf{w}) = 0$ for all $\mathsf{w} \notin L$.

定义 3.1 (识别)。固定一个字母表 Σ 。一个语言识别器是一个函数 $o : \Sigma^* \to \{0, 1\}$ 。一个语言 $L \subseteq \Sigma^*$ 被 o 识别，如果对于所有 $\mathsf{w} \in L$ 和对于所有 $\mathsf{w} \notin L$ ，有 $o(\mathsf{w}) = 1$ 和 $o(\mathsf{w}) = 0$ 。

We can also define string recognition over an EOS-extended alphabet $\overline{\Sigma} \stackrel{\text{def}}{=} \Sigma \cup \{\text{EOS}\}$, where $\text{EOS} \notin \Sigma$ is an end-of-sequence token.

我们还可以在扩展了 EOS 的字母表 $\overline{\Sigma} \stackrel{\text{def}}{=} \Sigma \cup \{\text{EOS}\}$ 上定义字符串识别，其中 $\text{EOS} \notin \Sigma$ 是一个序列结束标记。

Definition 3.2 (EOS-Recognition). Fix an alphabet Σ . Let $o : \overline{\Sigma}^* \to \{0, 1\}$ be a recognizer over the EOS-extended alphabet. We say that o EOS-recognizes a language $L \subseteq \Sigma^*$ if o recognizes the language $LEOS \stackrel{\text{def}}{=} \{\mathsf{w}\text{EOS} \mid \mathsf{w} \in L\} \subseteq \overline{\Sigma}^*$.

定义 3.2（EOS-识别）。固定一个字母表 Σ 。设 $o : \overline{\Sigma}^* \to \{0, 1\}$ 是在 EOS 扩展字母表上的识别器。我们说 o EOS-识别一种语言 $L \subseteq \Sigma^*$ ，如果 o 识别语言 $LEOS \stackrel{\text{def}}{=} \{\mathsf{w}\text{EOS} \mid \mathsf{w} \in L\} \subseteq \overline{\Sigma}^*$ 。

In other words, Definition 3.2 says that, given an input $\mathsf{w} = \mathsf{w}_1 \cdots \mathsf{w}_N \in \Sigma^*$, the recognizer

processes the EOS-padded string $\overline{\mathsf{w}} \stackrel{\text{def}}{=} \mathsf{w}_1 \cdots \mathsf{w}_N EOS$. This convention mirrors the semantics of LTL (see §A.3): a formula ψ is evaluated at position $N+1$, just beyond the last token of w . In the transformer, position $N+1$ corresponds to EOS.

换句话说，定义 3.2 表示，给定一个输入 $\mathsf{w} = \mathsf{w}_1 \cdots \mathsf{w}_N \in \Sigma^*$，识别器处理的是填充了 EOS 的字符串 $\overline{\mathsf{w}} \stackrel{\text{def}}{=} \mathsf{w}_1 \cdots \mathsf{w}_N EOS$。这种约定与 LTL 的语义相呼应（参见 §A.3）：一个公式 ψ 在位置 $N+1$ 进行评估，该位置位于 w 的最后一个标记之后。在 Transformer 中，位置 $N+1$ 对应于 EOS。

3.2 Attention

3.2 注意力

The Basic Definition. Attention can be viewed as a matrix-to-matrix function. Let $\mathbf{H} \in \mathbb{F}^{D \times (N+1)}$ be a matrix. We map H to another matrix as follows. We first define the linear transformations

基本定义。注意力可以被视为一个矩阵到矩阵的函数。设 $\mathbf{H} \in \mathbb{F}^{D \times (N+1)}$ 为一个矩阵。我们将 H 映射到另一个矩阵，如下所示。我们首先定义线性变换

$$\mathbf{Q}(\mathbf{H}) \stackrel{\text{def}}{=} \Theta^{\mathsf{Q}} \mathbf{H}, \tag{5a}$$

$$\mathbf{K}(\mathbf{H}) \stackrel{\text{def}}{=} \Theta^{\mathsf{K}} \mathbf{H}, \tag{5b}$$

$$\mathbf{V}(\mathbf{H}) \stackrel{\text{def}}{=} \Theta^{\mathsf{V}} \mathbf{H}, \tag{5c}$$

where $\Theta^{\mathsf{Q}} \in \mathbb{F}^{D_{\mathsf{K}} \times D}$, $\Theta^{\mathsf{K}} \in \mathbb{F}^{D_{\mathsf{K}} \times D}$, and $\Theta^{\mathsf{V}} \in \mathbb{F}^{D_{\mathsf{V}} \times D}$ are parameter matrices. These are often called the query, keys and values, respectively. For integers $n, m \in \{1, \ldots, N+1\}$, we then define the score as follows

where $\Theta^{\mathsf{Q}} \in \mathbb{F}^{D_{\mathsf{K}} \times D}$, $\Theta^{\mathsf{K}} \in \mathbb{F}^{D_{\mathsf{K}} \times D}$, and $\Theta^{\mathsf{V}} \in \mathbb{F}^{D_{\mathsf{V}} \times D}$ are parameter matrices. These are often called the query, keys and values, respectively. For integers $n, m \in \{1, \ldots, N+1\}$, we then define the score as follows

$$\mathbf{S}_{n,m}(\mathbf{H}) \stackrel{\text{def}}{=} \frac{\mathbf{Q}_{:,n}(\mathbf{H}) \cdot \mathbf{K}_{:,m}(\mathbf{H})}{\sqrt{D_{\mathsf{K}}}} \in \mathbb{F}. \tag{6}$$

Next, we define the α attention weights as

接下来，我们定义 α 注意力权重为

$$\alpha_{n,m}(\mathbf{H}) \stackrel{\text{def}}{=} \frac{\exp(\mathbf{S}_{n,m}(\mathbf{H}))\mathbf{M}_{n,m}}{\sum_{i=1}^{N+1} \exp(\mathbf{S}_{n,i}(\mathbf{H}))\mathbf{M}_{n,i}}, \tag{7}$$

where $\mathbf{M} \in \{0,1\}^{(N+1)\times(N+1)}$ is a binary matrix called the mask. Note that $\alpha_{n,m} \stackrel{def}{=} 0$ for all m if $M_{n,i} = 0$ for all i, i.e., if all preceding positions are masked. Finally, we define the resulting matrix

其中 $\mathbf{M} \in \{0,1\}^{(N+1)\times(N+1)}$ 是一个称为掩码的二进制矩阵。请注意，如果对于所有 m，$\alpha_{n,m} \overset{def}{=} 0$，即如果所有前面的位置都被掩码。最后，我们定义结果矩阵

$$\mathbf{O}_{:,n}(\mathbf{H}) \overset{\text{def}}{=} \sum_{m=1}^{N+1} \alpha_{n,m}(\mathbf{H})\mathbf{V}_{:,m}(\mathbf{H}), \tag{8}$$

which is the output of the function. In summary, we define the attention mechanism as follows

这就是该函数的输出。总之，我们定义注意力机制如下

$$\mathbf{A}\colon \mathbb{F}^{D\times(N+1)} \to \mathbb{F}^{D\times(N+1)}$$

$$\mathbf{H} \mapsto \mathbf{O}(\mathbf{H}). \tag{9}$$

Masking Patterns. We now give an exposition of several different choices of masks M. First, we define the global mask

遮罩模式。我们现在阐述几种不同的遮罩 M 的选择。首先，我们定义全局遮罩

$$\mathbf{M}^{*}_{n,m} \overset{\text{def}}{=} \begin{cases} 1 & \text{if} m < n, \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

which corresponds to global attention in the attention mechanism. Next, we define the k-local mask:

这对应于注意力机制中的全局注意力。接下来，我们定义 k 局部掩码：

$$\mathbf{M}^{\leq k}_{n,m} \overset{\text{def}}{=} \begin{cases} 1 & \text{if} \max(1, n-k) \leq m < n, \\ 0 & \text{otherwise}, \end{cases} \tag{11}$$

which corresponds to k -local attention.

这对应于 k -local 注意力。

Multi-Head Attention. It is also commonplace to combine attention heads. A multi-head attention with H heads $O^1, \ldots, O^H$ is defined below

多头注意力机制。通常也会将注意力头进行组合。具有 H 个注意力头的多头注意力机制 $O^1, \ldots, O^H$ 定义如下

$$\mathbf{A}(\mathbf{H}) \overset{\text{def}}{=} \sum_{h=1}^{H} \Theta^h \mathbf{O}^h(\mathbf{H}), \tag{12}$$

where the heads are combined by a linear projection, and each $\Theta^h \in \mathbb{F}^{D\times D_{\mathrm{v}}}$ is a parameter. Different heads may use different masks: for instance, some heads may use $\mathbf{M}^*$ while others use $\mathbf{M}^{\leq k}$, giving rise to hybrid global-local attention.

在头部通过线性投影进行组合，每个 $\Theta^h \in \mathbb{F}^{D \times D_{\mathrm{v}}}$ 是一个参数。不同的头部可能会使用不同的掩码：例如，一些头部可能使用 $\mathbf{M}^*$，而其他头部则使用 $\mathbf{M}^{\leq k}$，从而产生混合的全局-局部注意力机制。

3.3 Transformer Architecture

3.3 Transformer 架构

We now define a transformer recognizer.

我们现在定义一个变压器识别器。

Definition 3.3 (Transformer). Fix an alphabet Σ. A (D, L) -transformer is a 4-tuple composed of

定义 3.3（Transformer）。固定一个字母表 Σ。一个 (D, L) -transformer 是由以下四个部分组成的四元组

- a token encoder $\mathbf{e} : \overline{\Sigma} \to \mathbb{F}^D$,
- a family of attention mechanisms $\{\mathbf{A}^\ell\}_{\ell=1}^L$,
- a family of non-linear functions $\{\mathbf{F}^\ell\}_{\ell=1}^L$ where each $\mathbf{F}^\ell : \mathbb{F}^D \to \mathbb{F}^D$, and
- a classification function $c \colon \mathbb{F}^D \to \{0, 1\}$.
- 一个 token 编码器 $\mathbf{e} : \overline{\Sigma} \to \mathbb{F}^D$，
- 一组注意力机制 $\{\mathbf{A}^\ell\}_{\ell=1}^L$，
- 一组非线性函数 $\{\mathbf{F}^\ell\}_{\ell=1}^L$，其中每个 $\mathbf{F}^\ell : \mathbb{F}^D \to \mathbb{F}^D$，以及
- 一个分类函数 $c \colon \mathbb{F}^D \to \{0, 1\}$。

A transformer EOS-recognizes a language as follows. Given $\overline{\mathsf{w}} = \mathsf{w}_1 \cdots \mathsf{w}_N$ EOS where $\mathsf{w}_1, \ldots, \mathsf{w}_N \in \Sigma$, define the base case

变压器通过以下方式识别语言。给定 $\overline{\mathsf{w}} = \mathsf{w}_1 \cdots \mathsf{w}_N$ EOS，其中 $\mathsf{w}_1, \ldots, \mathsf{w}_N \in \Sigma$，定义基本情况

$$\mathbf{H}^0(\overline{\mathsf{w}})_{:,n} \stackrel{\text{def}}{=} \mathbf{e}(\mathsf{w}_n), \quad \forall n \in \{1, \ldots, N+1\}. \tag{13}$$

Then, for each layer $\ell = 1, \ldots, L$, compute

然后，对于每一层 $\ell = 1, \ldots, L$，计算

$$\mathbf{G}^\ell(\overline{\mathbf{w}}) \stackrel{\text{def}}{=} \mathrm{LN}\left(\mathbf{H}^{\ell-1}(\overline{\mathbf{w}}) + \mathbf{A}^\ell(\mathbf{H}^{\ell-1}(\overline{\mathbf{w}}))\right), \tag{14a}$$

$$\mathbf{H}^\ell(\overline{\mathbf{w}}) \stackrel{\text{def}}{=} \mathrm{LN}\left(\mathbf{G}^\ell(\overline{\mathbf{w}}) + \mathbf{F}^\ell(\mathbf{G}^\ell(\overline{\mathbf{w}}))\right), \tag{14b}$$

where $\mathbf{F}^{\ell}$ is applied column-wise, and LN is layer normalization (Ba et al., 2016). The transformer's output is then $o(\mathsf{w}) \stackrel{\text{def}}{=} c(\mathbf{H}^{L}(\overline{\mathsf{w}})_{:,N+1})$.

其中 $\mathbf{F}^{\ell}$ 是按列应用的，LN 是层归一化 (Ba 等，2016)。然后变压器的输出是 $o(\mathsf{w}) \stackrel{\text{def}}{=} c(\mathbf{H}^{L}(\overline{\mathsf{w}})_{:,N+1})$。

3.4 Fixed Precision

3.4 固定精度

In the exposition above, we assumed that transformers operate at fixed precision, i.e., there exists a finite set F of numbers—analogous to the floating-point formats used in practice (IEEE, 2019)—such that all parameters and intermediate values lie in F. Every primitive operation, including exp and the division in softmax, is performed in F and rounded back into F. We note that fixed-precision arithmetic differs from exact arithmetic in subtle ways: for instance, addition is not generally associative over F, so the order of summation in softmax can affect the result. Nevertheless, the fixed-precision assumption is standard in the analysis of transformers and is part of what makes the connection to finite automata and temporal logic possible.

在上面的阐述中，我们假设变换器以固定精度运行，即存在一个有限的数集 F——类似于实际中使用的浮点格式（IEEE, 2019）——使得所有参数和中间值都位于 F 中。每一个基本操作，包括 softmax 中的 exp 和除法，都是在 F 中进行并重新舍入回 F。我们注意到，固定精度算术与精确算术在某些细微之处有所不同：例如，在 F 上加法通常不是结合的，因此 softmax 中求和的顺序会影响结果。然而，固定精度的假设在变换器的分析中是标准的，也是使得与有限自动机和时序逻辑建立联系的要素之一。

4 Characterizing Transformers

4 描述变压器

We now show that global, local, and hybrid attention correspond exactly to the fragments introduced in §2. We first restate an existing result, relating transformers with global attention to LTL [P].

我们现在展示全球注意力、局部注意力和混合注意力与第 2 节中引入的片段完全对应。我们首先重述一个现有的结果，将具有全局注意力的变压器与 LTL [P] 相关联。

Theorem 4.1 (Li and Cotterell, 2025). A language L is EOS-recognizable by a (D, L) -transformer

定理 4.1 (Li 和 Cotterell, 2025)。一种语言 L 可以由一个 (D, L) -transformer 进行 EOS-可识别

$(\mathbf{e}, \{\mathbf{A}^{\ell}\}_{\ell=1}^{L}, \{\mathbf{F}^{\ell}\}_{\ell=1}^{L}, c)$ in which every attention head uses the global mask $\mathbf{M}^{*}$ if and only if it is definable in LTL [P].

$(\mathbf{e}, \{\mathbf{A}^{\ell}\}_{\ell=1}^{L}, \{\mathbf{F}^{\ell}\}_{\ell=1}^{L}, c)$ 中的每个注意力头在且仅当其可在 LTL [P] 中定义时使用全局掩码 $\mathbf{M}^{*}$。

We now consider k -local attention.

我们现在考虑 k -local 注意力。

Theorem 4.2. A language L is EOS-recognizable by $a(D, L)$ -transformer $(\mathbf{e}, \{\mathbf{A}^{\ell}\}_{\ell=1}^{L}, \{\mathbf{F}^{\ell}\}_{\ell=1}^{L}, c)$ in which every attention head uses the k -local mask $\mathbf{M}^{\leq k}$ if and only if it is definable in LTL $[\mathbf{Y}^{\leq k}]$.

定理 4.2. 一种语言 L 可以由 $a(D, L)$ -transformer $(\mathbf{e}, \{\mathbf{A}^{\ell}\}_{\ell=1}^{L}, \{\mathbf{F}^{\ell}\}_{\ell=1}^{L}, c)$ 以 EOS 可识别，当且

仅当该语言在 LTL $[\mathbf{Y}^{\leq k}]$ 中可定义，其中每个注意力头使用 k -local mask $\mathbf{M}^{\leq k}$ 。

Proof sketch. The argument follows the same structure as Li and Cotterell (2025) for Theorem 4.1. Because each query attends to at most k predecessors, all summations in the masked softmax and value aggregation reduce to bounded counting, which is expressible in LTL $[Y \leq k]$ using bounded nesting of $Y \leq k$. Conversely, by induction on LTL $[Y \leq k]$ formulas, Boolean connectives are implemented positionwise, and $Y \leq k\psi$ is realized by a k-local attention head that aggregates information from the last k positions satisfying ψ . The same refinement step as in Li and Cotterell (2025, Lemma B.12) can be applied.

证明草图。论证的结构与 Li 和 Cotterell（2025）在定理 4.1 中的结构相同。由于每个查询最多关注 k 个前缀，因此在掩码 softmax 和值聚合中的所有求和都可归约为有界计数，这可以通过 LTL $[Y \leq k]$ 使用 $Y \leq k$ 的有界嵌套来表示。相反地，通过在 LTL $[Y \leq k]$ 公式上进行归纳，布尔连接词在位置上被实现，并且 $Y \leq k\psi$ 通过一个 k 局部注意力头实现，该注意力头从满足 ψ 的最后 k 个位置中聚合信息。可以应用与 Li 和 Cotterell（2025，引理 B.12）中相同的细化步骤。

Finally, we consider hybrid transformers that mix global and k -local heads.

最后，我们考虑混合变压器，它们结合了全局和 k -局部头。

Theorem 4.3. A language L is EOS-recognizable by a (D, L) -transformer $(\mathbf{e}, \{\mathbf{A}^{\ell}\}_{\ell=1}^{L}, \{\mathbf{F}^{\ell}\}_{\ell=1}^{L}, c)$ in which some heads use $\mathbf{M}^{*}$ and the remaining heads use $\mathbf{M}^{\leq k}$ if and only if it is definable in LTL $[\mathbf{P}, \mathbf{Y}^{\leq k}]$.

定理 4.3。一个语言 L 可以由一个 (D, L) -转换器 $(\mathbf{e}, \{\mathbf{A}^{\ell}\}_{\ell=1}^{L}, \{\mathbf{F}^{\ell}\}_{\ell=1}^{L}, c)$ 进行 EOS-可识别，其中某些头使用 $\mathbf{M}^{*}$，其余头使用 $\mathbf{M}^{\leq k}$，当且仅当它可以在 LTL $[\mathbf{P}, \mathbf{Y}^{\leq k}]$ 中定义。

Proof. The theorem is a direct corollary of Theorems 4.1 and 4.2.

证明。该定理是定理 4.1 和 4.2 的直接推论。

4.1 Outlook

4.1 Outlook

We now translate the logical characterizations into concrete consequences for transformers under different attention patterns. An immediate consequence of Theorems 4.1 to 4.3 together with Corollary 2.10 is that transformers with hybrid global-local attention are strictly more expressive than either global-only or k -local-only transformers. Thus, mixing local and global attention is not only computationally attractive (Luong et al., 2015; Child et al., 2019; Beltagy et al., 2020; Zaheer et al., 2020), but can also yield a strict expressive gain. Moreover, 1-local attention is the strongest choice within the local-attention family: by Propositions 2.11 and 2.12 and Theorems 4.2 and 4.3, for every $k > 1$, transformers with 1-local attention are strictly more expressive than those with k -local attention, and the same holds when global heads

我们现在将逻辑上的特性描述转化为不同注意力模式下变压器的具体后果。定理 4.1 到 4.3 与推论 2.10 的直接后果是，具有混合全局-局部注意力的变压器比仅全局注意力或 k -局部注意力的变压器具有更强的表达能力。因此，混合局部和全局注意力不仅在计算上具有吸引力（Luong 等，2015；Child 等，2019；Beltagy 等，2020；Zaheer 等，2020），而且还能带来严格的表达能力提升。此外，1-局部注意力是在局部注意力家族中最强的选择：根据命题 2.11 和 2.12 以及定理 4.2 和 4.3，对于每一个 $k > 1$ ，具有 1-局部注意力的变压器比具有 k -局部注意力的变压器具有更强的表达能力，当使用全局头时也是如此。

are added. This suggests that 1-local attention is a particularly promising choice in practice. We note, however, two important caveats.

被添加了。这表明在实践中，1-local 注意力是一个特别有前景的选择。然而，我们注意到两个重要的注意

事项。

Depth Overhead. Although 1-local attention maximizes expressivity, replacing a single k-local head by 1-local heads may incur a depth overhead. At the logical level, expressing $Y^{\leq k}\psi$ via Eq. (3a) requires a disjunction of formulas of the form $Y^i\psi$ for $1 \leq i \leq k$. Thus, using only Y increases the operator depth (§ A.2) from $\mathrm{od}(\psi) + 1$ to as much as $\mathrm{od}(\psi) + k$. In other words, eliminating $Y^{\leq k}$ in favor of Y preserves expressivity, but may increase the required depth by an overhead of up to k - 1. Specifically, in the proof of Theorem 4.2, each temporal operator requires at least one layer to simulate, so this translation may require up to k stacked layers in place of a single layer. Thus, under a fixed depth, transformers with 1-local attention need not be strictly more expressive than transformers with k-local attention. In practice, modern language models are deep (OpenAI, 2023), making this limitation less of a practical detriment and rendering architectures with 1-local heads a plausible way to increase expressivity.

深度开销。虽然 1-local 注意力最大化了表达能力，但用 1-local 头替换一个 k-local 头可能会导致深度开销。在逻辑层面，通过公式 (3a) 表达 $Y^{\leq k}\psi$ 需要形式为 $Y^i\psi$ 的公式的析取，对于 $1 \leq i \leq k$。因此，仅使用 Y 会增加操作符深度（§ A.2）从 $\mathrm{od}(\psi) + 1$ 增加到最多 $\mathrm{od}(\psi) + k$。换句话说，用 Y 替代 $Y^{\leq k}$ 保持了表达能力，但可能会增加所需的深度，最多增加 k - 1 的开销。具体来说，在定理 4.2 的证明中，每个时间操作符至少需要一层来模拟，因此这种转换可能需要最多 k 层堆叠，而不是单层。因此，在固定深度下，具有 1-local 注意力的变压器并不一定比具有 k-local 注意力的变压器更具表达力。实际上，现代语言模型是深度的（OpenAI, 2023），这使得这种限制在实践中不太成问题，并使得具有 1-local 头的架构成为增加表达能力的一种合理方式。

Positional Encodings. The strict separation above holds in the absence of positional encodings. With sufficiently rich positional information, however, the gap can disappear. Intuitively, if the model can always identify the immediate predecessor within a k-window, then k-local attention can simulate 1-local attention. Positional encodings are often viewed as numerical predicates added to the logic (Yang et al., 2024; Li and Cotterell, 2025). By Proposition 2.13, suitable modular predicates suffice to erase the gap. Therefore, if positional encodings that simulate such predicates are available, then a k-local head can recover the immediate predecessor by selecting, at each position, the unique position in the previous k steps. Interestingly, sinusoidal encodings (SiPE; Vaswani et al. 2017) and rotary encodings (RoPE; Su et al. 2024) can be related to modular predicates (Yang et al., 2025). However, realizing modular predicates in this way requires a rational variant (Chiang et al., 2023), which is not the form typically used in practice.

位置编码。上述严格的分离在没有位置编码的情况下成立。然而，当具有足够丰富的位置信息时，这种差距可以消失。直觉上，如果模型总能在一个 k 窗口内识别出前一个元素，那么 k 局部注意力可以模拟 1 局部注意力。位置编码通常被视为添加到逻辑中的数值谓词（Yang 等人，2024；Li 和 Cotterell，2025）。根据命题 2.13，合适的模运算谓词足以消除这种差距。因此，如果存在能够模拟此类谓词的位置编码，那么一个 k 局部头可以通过在每个位置选择前 k 步中的唯一位置来恢复前一个元素。有趣的是，正弦编码（SiPE；Vaswani 等人，2017）和旋转编码（RoPE；Su 等人，2024）可以与模运算谓词相关联（Yang 等人，2025）。然而，以这种方式实现模运算谓词需要一个有理数变体（Chiang 等人，2023），而这并不是实践中通常使用的形式。

5 Empirically Verifying the Theory

5 实证验证该理论

In this section, we test the expressivity predictions of our theory on formal language recognition tasks.

在本节中，我们测试了我们的理论对形式语言识别任务的表达能力预测。

5.1 Languages

5.1 语言

We evaluate a suite of formal languages grouped by definability in temporal-logic fragments, selecting two representative languages from each class:

我们评估了一组按时间逻辑片段中可定义性分类的形式语言，从每个类别中选择两种代表性语言：

- LTL [Y]-definable: $\Sigma^* a$ (strings ending with a) and $\Sigma^* ab$ (strings ending with ab).
- LTL [P]-definable: $a\Sigma^*$ (strings starting with a) and $\Sigma^* a \Sigma^* b \Sigma^*$ (strings containing ab as a not-necessarily-contiguous subsequence).
- LTL [P, Y]-definable but not in LTL [P] or LTL [Y]: (ab)* (strings of repeated ab) and Σ^* ab Σ^* (strings containing ab as a contiguous substring).
- LTL[S]-definable but not in LTL [P, Y]: $\{a, b, d\}^* a \{c, d\}^*$ (a right-deterministic polynomial) and $(a(ab)^* b)^*$ (bounded Dyck with maximum nesting depth 2).

- LTL [Y]-definable: $\Sigma^* a$ (strings ending with a) and $\Sigma^* ab$ (strings ending with ab).
- LTL [P]-definable: $a\Sigma^*$ (strings starting with a) and $\Sigma^* a \Sigma^* b \Sigma^*$ (strings containing ab as a not-necessarily-contiguous subsequence).
- LTL [P, Y]-definable but not in LTL [P] or LTL [Y]: (ab)* (strings of repeated ab) and Σ^* ab Σ^* (strings containing ab as a contiguous substring).
- LTL[S]-definable but not in LTL [P, Y]: $\{a, b, d\}^* a \{c, d\}^*$ (a right-deterministic polynomial) and $(a(ab)^* b)^*$ (bounded Dyck with maximum nesting depth 2).

5.2 Experimental Setup

5.2 实验设置

We consider three attention settings, corresponding to the masking patterns defined in §3: local-only (all heads use $M^{\leq k}$), hybrid (half the heads use $M^{\leq k}$, the rest use M^*), and global-only (all heads use M^*). For local-only and hybrid models, we vary the window size $k \in \{1, 2, 4\}$. We also evaluate transformers with two positional encodings: sinusoidal encodings (SiPE; Vaswani et al. 2017) and rotary encodings (RoPE; Su et al. 2024). The models are trained on strings of length up to 40 and evaluated on lengths 41–500. Each experiment is run with 5 random seeds and 3 learning rates; additional details are given in §D.2. In addition to accuracy, we report the longest perfect length, defined as the largest length up to which the model achieves 100% accuracy.

我们考虑三种注意力设置，对应于第 3 节中定义的掩码模式：局部注意力（所有头使用 $M^{\leq k}$），混合注意力（一半的头使用 $M^{\leq k}$，其余使用 M^*），以及全局注意力（所有头使用 M^*）。对于局部注意力和混合注意力模型，我们变化窗口大小 $k \in \{1, 2, 4\}$。我们还评估了两种位置编码的变压器：正弦编码（SiPE；Vaswani 等，2017）和旋转编码（RoPE；Su 等，2024）。模型在长度最多为 40 的字符串上进行训练，并在长度 41 至 500 的字符串上进行评估。每个实验使用 5 个随机种子和 3 个学习率进行运行；更多细节请参见第 D.2 节。除了准确率外，我们还报告了最长完美长度，定义为模型达到 100% 准确率的最大长度。

5.3 Results

5.3 结果

We summarize performance using a heatmap of the longest perfect length in Fig. 3. Full numerical results, including accuracies and longest perfect lengths, are provided in §D.3.

我们使用图 3 中的热图总结性能，其中显示了最长完美长度。完整的数值结果，包括准确率和最长完美长度，详见 §D.3。

5.3.1 Results on NoPE

5.3.1 无 PE 结果

We begin with a discussion of the NoPE setting, whose results are shown on the left panel of Fig. 3.

我们首先讨论 NoPE 设置，其结果如图 3 的左侧面板所示。

Local-only models match LTL [Y]. As predicted, local-only transformers succeed precisely on the LTL [Y]-definable languages: they achieve perfect generalization up to length 500 on the two LTL [Y] tasks in Fig. 3, while failing to generalize on the remaining languages.

本地模型与 LTL [Y] 匹配。正如预测的那样，仅本地的变压器在 LTL [Y] 可定义的语言上表现出色：它们在图 3 中两个 LTL [Y] 任务上实现了长达 500 的完美泛化，而在其他语言上则无法泛化。

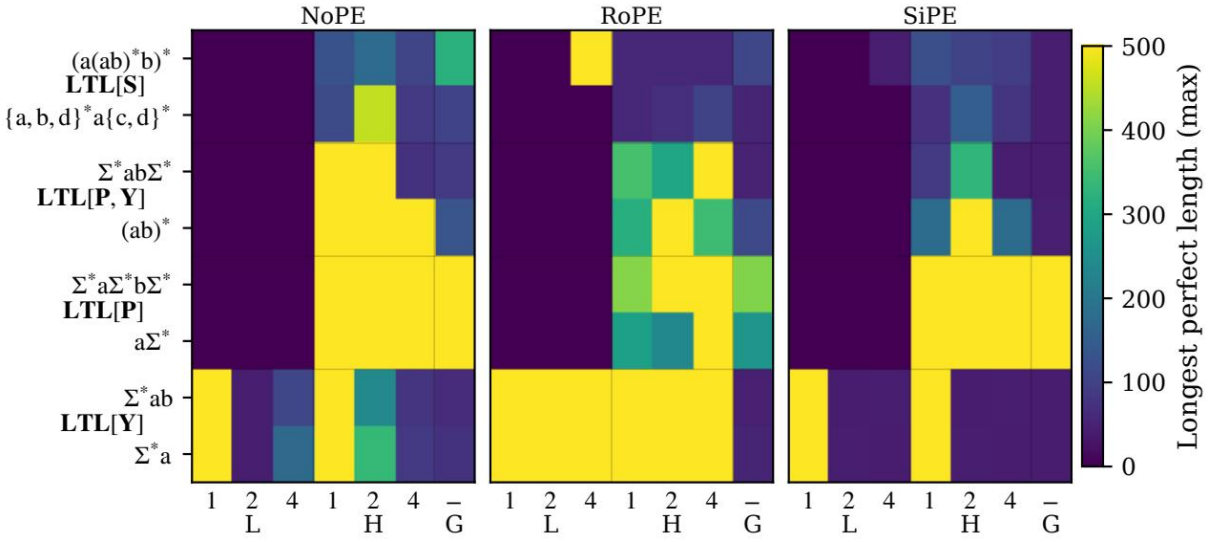

Figure 8: images/c466a2c5bce81db23b37412475f2da228b95fb905c5ef5ae4b86905ff6f5abba.jpg

Figure 3: Heatmap of longest perfect lengths (maximum over runs) across formal languages, attention patterns, and positional encodings. Columns denote local (L), hybrid (H), and global (G) attention patterns; for L and H, the labels above columns indicate the local window size k .

图 3：在形式语言、注意力模式和位置编码中，最长完美长度（最大运行值）的热图。列表示局部（L）、混合（H）和全局（G）注意力模式；对于 L 和 H，列上方的标签表示局部窗口大小 k 。

Global-only models match LTL [P]. Conversely, global-only models succeed only on the LTL [P]-

definable languages and do not exhibit perfect length generalization beyond this class.

全局模型匹配 LTL [P]。相反，全局模型只能在 LTL [P] 可定义的语言上取得成功，并且在超出这一类的语言上无法表现出完美的长度泛化能力。

Hybrid models match LTL [P, Y]. Hybrid models, especially with k = 1, succeed on all languages definable in LTL [P, Y], which subsumes both LTL [P] and LTL [Y]. As visualized in Fig. 3, they generalize perfectly up to length 500 on the bottom six languages.

混合模型匹配 LTL [P, Y]。特别是当 k = 1 时，混合模型在所有可由 LTL [P, Y] 定义的语言上都能成功，这涵盖了 LTL [P] 和 LTL [Y]。如图 3 所示，它们在底部六个语言上完美地推广到长度 500。

1-local attention. Among the tested window sizes, k = 1 is consistently the strongest performer. Both k = 2 and k = 4 fail even on definite languages, and increasing the window can hurt: for example, hybrid models with k = 2 learn $\Sigma^* ab \Sigma^*$, whereas those with k = 4 do not.

1-局部注意力。在测试的窗口大小中，k = 1 始终是表现最强的。k = 2 和 k = 4 甚至在确定性语言上也表现不佳，增大窗口可能会带来负面影响：例如，使用 k = 2 的混合模型可以学习 $\Sigma^* ab \Sigma^*$，而使用 k = 4 的模型则无法做到这一点。

Beyond LTL [P, Y]. As expected, none of the models perfectly generalizes on the two languages in LTL[S] LTL [P, Y], namely the right-deterministic polynomial and bounded Dyck. Nevertheless, hybrid attention yields the strongest partial generalization on these tasks.

Beyond LTL [P, Y]. 如预期的那样，没有任何模型在 LTL[S] LTL [P, Y] 中的两种语言上完美地泛化，即右确定性多项式和有界 Dyck 语言。然而，混合注意力在这些任务上实现了最强的局部泛化。

5.3.2 Positional Encodings

5.3.2 位置编码

We now turn to the settings with positional encodings (middle and right panels of Fig. 3).

我们现在转向具有位置编码的设置（见图 3 的中间和右侧小图）。

Global-only models. One might hope that positional encodings would allow global attention to approximate locality. In our experiments, however, neither SiPE nor RoPE compensates for the absence of local attention: Positional encodings do not erase

全局模型。人们或许希望位置编码能够让全局注意力近似局部性。然而在我们的实验中，SiPE 或 RoPE 并未弥补局部注意力的缺失：位置编码并不会消除这种影响。

the qualitative separation between global-only and hybrid models; on the contrary, they often reduce performance on the tested languages.

全球模型与混合模型之间的定性差异；相反，它们经常导致测试语言上的性能下降。

Local-only and hybrid models. With RoPE, the gap between 1-local and larger-window local attention becomes smaller. In particular, local-only models with k = 2 or k = 4 can now recognize $\Sigma^* a$ and $\Sigma^* ab$. In hybrid models, RoPE also enables k = 4 to recognize $\Sigma^* ab \Sigma^*$. However, these gains come with tradeoffs, as RoPE degrades performance in several other settings. By comparison, SiPE does not narrow the gap and instead tends to reduce performance across the board.

本地模型和混合模型。使用 RoPE，1 本地模型与更大窗口本地注意力之间的差距变得更小。特别是，使用 k = 2 或 k = 4 的纯本地模型现在可以识别 $\Sigma^* a$ 和 $\Sigma^* ab$ 。在混合模型中，RoPE 也使 k = 4 能够识别 $\Sigma^* ab \Sigma^*$ 。然而，这些收益伴随着权衡，因为 RoPE 在其他一些场景中会降低性能。相比之下，SiPE 并不会缩小这个差距，反而倾向于全面降低性能。

Summary. Overall, on one hand, the results under NoPE align exactly with our theory. On the other hand, the role of positional encodings is less clear-cut and merits further investigation.

摘要。总体而言，一方面，NoPE 的结果与我们的理论完全一致。另一方面，位置编码的作用尚不明确，值得进一步研究。

6 An Experiment on Natural Language

6 一个自然语言实验

We also experiment with whether our theory has something to say about natural language. Thus, we run a lightweight next-token language modeling experiment on WikiText-2.

我们还尝试检验我们的理论是否对自然语言有所启示。因此，我们在 WikiText-2 上运行了一个轻量级的下一个词语言建模实验。

6.1 Experimental Setup

6.1 实验设置

We train a GPT-2-small-style decoder-only Transformer from scratch, using the standard GPT-2 small architecture (12 layers, hidden size 768, 12 attention heads, and feed-forward size 3072). We keep the architecture and optimization setup fixed across all runs, varying only the attention pattern.

我们从头开始训练一个类似于 GPT-2-small 的解码器-only Transformer，使用标准的 GPT-2 small 架构（12 层，隐藏大小 768，12 个注意力头，以及前馈大小 3072）。我们在所有运行中保持架构和优化设置不变，仅改变注意力模式。

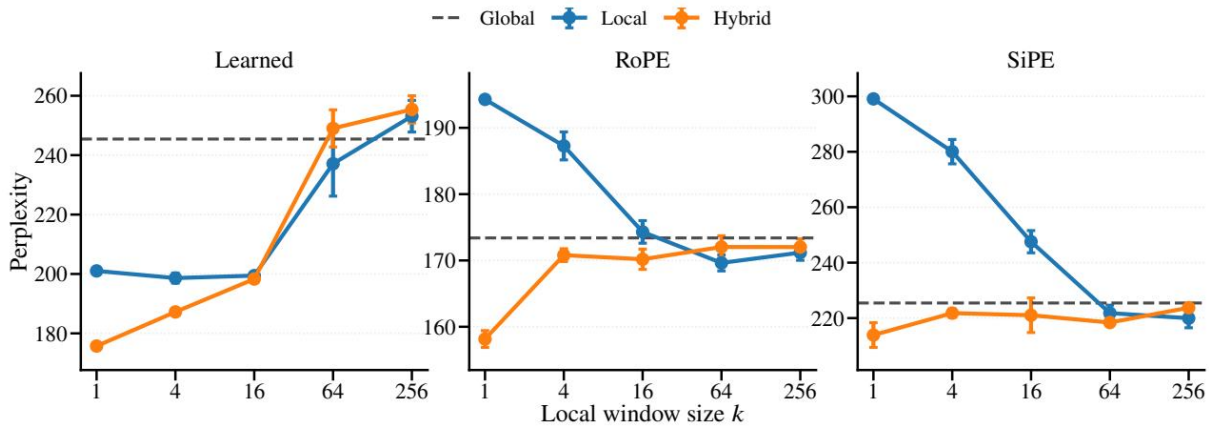

Figure 9: images/137b28069d1d8ddffb0df360b5c898dea525b59e65f68cf7d4b03550002c72a7.jpg

Figure 4: Perplexity on WikiText-2 for local, hybrid, and global attention patterns under different positional encodings. Curves show mean perplexity across runs for local and hybrid models as a function of the local window size k, and error bars indicate standard deviations. The dashed horizontal line shows the corresponding global baseline.

图 4：在 WikiText-2 数据集上，针对局部、混合和全局注意力模式在不同位置编码下的困惑度。曲线显示了局部和混合模型在局部窗口大小 k 变化时的平均困惑度，误差棒表示标准差。虚线水平线表示相应的全局基线。

Concretely, we compare global-only, local-only, and hybrid attention, where in the hybrid setting half of the heads are replaced by local heads. To remain consistent with the formal setup in §3, we use strict causal masking throughout. We consider local window sizes $k \in \{1, 4, 16, 64, 256\}$ and three positional encoding schemes: learned absolute positional embeddings, RoPE, and SiPE. For optimization, we use AdamW (Loshchilov and Hutter, 2019) with learning rate 2.5×10^{-4}, weight decay 0.01, 2000 warmup steps, cosine learning-rate decay, per-device batch size 2, gradient accumulation of 32 steps, and early stopping based on validation loss.

具体来说，我们比较了仅使用全局注意力、仅使用局部注意力以及混合注意力的设置，在混合设置中，有一半的注意力头被替换为局部注意力头。为了与第 3 节中的正式设置保持一致，我们在整个过程中都使用严格的因果掩码。我们考虑了局部窗口大小 $k \in \{1, 4, 16, 64, 256\}$ 和三种位置编码方案：学习的绝对位置嵌入、RoPE 和 SiPE。在优化方面，我们使用 AdamW（Loshchilov 和 Hutter, 2019）进行优化，学习率设置为 2.5×10^{-4}，权重衰减为 0.01，预热步数为 2000 步，学习率采用余弦衰减，每个设备的批量大小为 2，梯度累积步数为 32 步，并基于验证损失进行早停。

6.2 Results

6.2 结果

The results show a clear and consistent pattern. Across all positional encoding choices, the hybrid model is always the strongest, and within the hybrid family, k = 1 is always the best setting. Hybrid attention with k = 1 reduces perplexity by 69.7, 15.2, 11.5 relative to the global-only baseline under various positional encodings. Increasing the local window generally degrades performance, and with sufficiently large windows the hybrid model can match or underperform the global-only baseline.

结果显示出一个清晰且一致的模式。在所有位置编码选择中，混合模型始终是最强的，而在混合模型家族中，k = 1 始终是最好的设置。使用 k = 1 的混合注意力，在各种位置编码下，相比仅全局的基线模型，可以将困惑度降低 69.7、15.2、11.5。增加局部窗口通常会降低性能，并且当窗口足够大时，混合模型可以与仅全局的基线模型相匹配或表现更差。

We also find that local-only attention can outperform global-only attention, which is surprising but compatible with our theory, since local-only and global-only attention are incomparable in expressive power. One possible explanation is that WikiText-2 relies more heavily on short-range patterns than on the longer-range dependencies. The effect of window size, however, depends on the positional encoding. Under learned positional embeddings, smaller local windows perform better. Under

我们还发现，仅局部注意力机制可以优于仅全局注意力机制，这令人惊讶但与我们的理论是一致的，因为局部注意力和全局注意力在表达能力上是不可比较的。一个可能的解释是，WikiText-2 更多地依赖于短距离模式，而不是更长距离的依赖关系。然而，窗口大小的影响取决于位置编码。在学习得到的位置嵌入下，较小的局部窗口表现更好。在

RoPE and SiPE, local-only models instead tend to prefer larger windows. This is consistent with the theory: although 1-local attention is maximally expressive in the unbounded setting, its advantage may disappear under a fixed depth bound, especially when suitable positional encodings narrow the gap between different local window sizes.

RoPE 和 SiPE 是局部模型，它们更倾向于使用较大的窗口。这与理论是一致的：虽然 1-local 注意力在无限制的设置下具有最大的表达能力，但在固定深度限制下，其优势可能会消失，尤其是在合适的 positional encodings 缩小了不同局部窗口大小之间的差距时。

Overall, our results support the main claim of the paper: local attention is complementary to global attention, and the hybrid setting combines the strengths of both. In particular, pairing global attention with 1-local attention yields the largest additional benefit.

总体而言，我们的结果支持了论文的主要观点：局部注意力与全局注意力是互补的，混合设置结合了两者

的优点。特别是，将全局注意力与 1-局部注意力结合，能够带来最大的额外收益。

7 Conclusion

7 结论

We have given a formal account of why local attention helps in transformer language models. By connecting attention patterns to fragments of linear temporal logic, we showed that local and global attention are complementary rather than interchangeable, and that adding local attention to a globally attentive transformer strictly increases expressive power. In particular, global attention alone corresponds to the logic LTL [P], purely k-local attention corresponds to LTL $\left[Y^{\leq k}\right]$, and hybrid global–local attention corresponds to the richer logic LTL $\left[P, Y^{\leq k}\right]$. The largest gain arises from 1-local attention, which induces the Y operator and captures, among other classes, the locally testable languages—a classical family beyond the reach of global-only models. Our formal-language experiments empirically confirm these separations, and a preliminary language-modeling experiment on WikiText-2 shows that hybrid attention with k=1 consistently achieves the best performance.

我们给出了一个正式的解释，说明局部注意力为何有助于变压器语言模型。通过将注意力模式与线性时态逻辑的片段联系起来，我们展示了局部注意力和全局注意力是互补而非可互换的，并且在全局注意力的变压器中添加局部注意力会严格增加表达能力。特别是，仅全局注意力对应于逻辑 LTL [P]，纯粹的 k 局部注意力对应于 LTL $\left[Y^{\leq k}\right]$，而混合的全局-局部注意力对应于更丰富的逻辑 LTL $\left[P, Y^{\leq k}\right]$。最大的增益来自于 1 局部注意力，它引入了 Y 运算符，并捕捉了包括局部可测试语言在内的多个类别——这是一个超越仅全局模型能力的经典语言家族。我们的形式语言实验实证地确认了这些分离，并且在 WikiText-2 上进行的初步语言建模实验表明，k=1 的混合注意力始终取得最佳性能。

Limitations

限制

Due to limited computational resources, we are unable to run large-scale language modeling experiments. Instead, we use a lightweight setup that is intended as a sanity check while remaining manageable in terms of model size, dataset, and training budget. Accordingly, our natural-language results should be interpreted with caution. It is possible that the qualitative picture changes with larger models, longer training, or broader datasets.

由于计算资源有限，我们无法运行大规模语言建模实验。相反，我们使用了一种轻量级的设置，旨在作为初步验证，同时在模型规模、数据集和训练预算方面保持可控。因此，我们自然语言方面的结果应谨慎解读。有可能在使用更大的模型、更长的训练时间或更广泛的数据集时，定性结果会发生变化。

Ethical Considerations

伦理考量

We do not foresee any ethical issues.

我们不预见到任何伦理问题。

References

参考资料

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer normalization. In NIPS Deep Learning Symposium.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In The Third International Conference on Learning Representations.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. Computing Research Repository, arXiv:2004.05150. Version 2.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In Advances in Neural Information Processing Systems, volume 33, pages 1877–1901. Curran Associates, Inc.

Janusz Brzozowski, Baiyu Li, and David Liu. 2012. Syntactic complexities of six classes of star-free languages. Journal of Automata, Languages and Combinatorics, 17(2):83–105.

Alexandra Butoi, Ghazal Khalighinejad, Anej Svete, Josef Valvoda, Ryan Cotterell, and Brian DuSell. 2025. Training neural networks as recognizers of formal languages. In The Thirteenth International Conference on Learning Representations.

David Chiang, Peter Cholak, and Anand Pillay. 2023. Tighter bounds on the expressivity of transformer encoders. In Proceedings of the 40th International Conference on Machine Learning, volume 202 of Proceedings of Machine Learning Research, pages 5544–5562. PMLR.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. Computing Research Repository, arXiv:1904.10509.

Joëlle Cohen, Dominique Perrin, and Jean-Eric Pin. 1993. On the expressive power of temporal logic. Journal of Computer and System Sciences, 46(3):271–294.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. Computing Research Repository, arXiv:2501.12948.

Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A Ortega. 2023. Neural networks and the chomsky hierarchy. In The Eleventh International Conference on Learning Representations.

Samuel Eilenberg. 1974. Automata, Languages, and Machines. Number pt. 2 in 59/B. Academic Press.

Dov Gabbay, Amir Pnueli, Saharon Shelah, and Jonathan Stavi. 1980. On the temporal analysis of fairness. POPL '80, page 163–173, New York, NY, USA. Association for Computing Machinery.

James A. Green. 1951. On the structure of semigroups. Annals of Mathematics, 54(1):163-172.

John Hewitt, Michael Hahn, Surya Ganguli, Percy Liang, and Christopher D. Manning. 2020. RNNs can generate bounded hierarchical languages with optimal memory. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1978–2010, Online. Association for Computational Linguistics.

IEEE. 2019. IEEE standard for floating-point arithmetic. IEEE Std 754-2019.

Johan Anthony Wilem Kamp. 1968. Tense Logic and the Theory of Linear Order. Ph.D. thesis, University of California, Los Angeles.

Diederik P. Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. Computing Research Repository, arXiv:1412.6980. Version 9.

Stephen C. Kleene. 1956. Representation of Events in Nerve Nets and Finite Automata, pages 3–42. Princeton University Press, Princeton.

Jiaoda Li and Ryan Cotterell. 2025. Characterizing the expressivity of fixed-precision transformer language models. In The Thirty-ninth Annual Conference on Neural Information Processing Systems.

Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. 2024. Chain of thought empowers transformers to solve inherently serial problems. In The Twelfth International Conference on Learning Representations.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. 层归一化。在 NIPS 深度学习研讨会。
Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. 通过联合对齐和翻译学习来实现神经机器翻译。在第三届国际学习表示会议。
Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer：长文档的变压器。计算研究仓库，arXiv:2004.05150。版本 2。
Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, 和 12 其他人。2020. 语言模型是少样本学习者。在《神经信息处理系统进展》，第 33 卷，第 1877–1901 页。Curran Associates, Inc。
Janusz Brzozowski, Baiyu Li, and David Liu. 2012. 六类无星语言的句法复杂性。自动机、语言与组合学杂志，17(2):83–105。
Alexandra Butoi, Ghazal Khalighinejad, Anej Svete, Josef Valvoda, Ryan Cotterell, and Brian DuSell. 2025. 用神经网络作为形式语言识别器进行训练。在第十三届国际学习表示会议。
David Chiang, Peter Cholak, and Anand Pillay. 2023. 变压器编码器表达力的更紧上限。在第 40 届国际机器学习会议论文集，第 202 卷，第 5544–5562 页。PMLR。
Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. 使用稀疏变压器生成长序列。计算研究仓库，arXiv:1904.10509。
Joëlle Cohen, Dominique Perrin, and Jean-Eric Pin. 1993. 时态逻辑的表达能力。计算机与系统科学杂志，46(3):271–294。
DeepSeek-AI. 2025. DeepSeek-R1：通过强化学习激励大语言模型的推理能力。计算研究仓库，arXiv:2501.12948。
Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, 和 Pedro A Ortega。2023. 神经网络与乔姆斯基层次结构。在第十一届国际学习表示会议。
Samuel Eilenberg. 1974. 自动机、语言和机器。第 59/B 卷第 2 部分。学术出版社。
Dov Gabbay, Amir Pnueli, Saharon Shelah, 和 Jonathan Stavi. 1980. 公平性的时间分析。POPL '80，第 163–173 页，纽约，美国。计算机协会。
James A. Green. 1951. 半群的结构。数学年鉴，54(1):163-172。
John Hewitt, Michael Hahn, Surya Ganguli, Percy Liang, 和 Christopher D. Manning. 2020. RNN 可以生成具有最优记忆的有界层次语言。在 2020 年自然语言处理经验方法会议（EMNLP）论文集，第 1978–2010 页，线上。计算语言学协会。
IEEE. 2019. 浮点运算的 IEEE 标准。IEEE 标准 754-2019。
Johan Anthony Wilem Kamp. 1968. 时间逻辑与线性顺序理论。加州大学洛杉矶分校博士论文。
Diederik P. Kingma 和 Jimmy Ba. 2014. Adam：一种随机优化方法。计算研究仓库，arXiv:1412.6980。版本 9。
Stephen C. Kleene. 1956. 神经网络和有限自动机中的事件表示，第 3–42 页。普林斯顿大学出版社，普林斯顿。
Jiaoda Li 和 Ryan Cotterell. 2025. 固定精度变压器语言模型的表达能力特征。在第 39 届神经信息处理系统年度会议。
Zhiyuan Li, Hong Liu, Denny Zhou, 和 Tengyu Ma. 2024. 思维链使变压器能够解决本质上串行的问题。在第十二届国际学习表示会议。